Real-Time AI Prediction Platform with Automated MLOps Pipeline

---

GitHubRepo : https://github.com/ChandikaGanewatte/Code_Review_Assitant_P.git
Deploy App Link : https://codereviewassitantp-mx6skijk3uqf6bnvmwbsew.streamlit.app/

Chandika Ganewatte
Data Science Intern - Gamage Recruiters
June 2026

– Content –

1. Executive Summary

This document outlines the design, development, and deployment of a production-grade MLOps platform engineered for retail sales prediction. The system addresses the critical business need for adaptive machine learning models that evolve with changing data patterns.

The solution implements a continuous integration and continuous deployment (CI/CD) pipeline, automating the transition from raw data ingestion to model serving. Key features include real-time inference via FastAPI, automated data drift monitoring using Kolmogorov–Smirnov (K-S) testing, and self-healing capabilities via automated model retraining pipelines managed by MLflow.

2. System Architecture

The platform follows a modular microservices architecture designed for scalability and maintainability.

2.1 High-Level Component Diagram

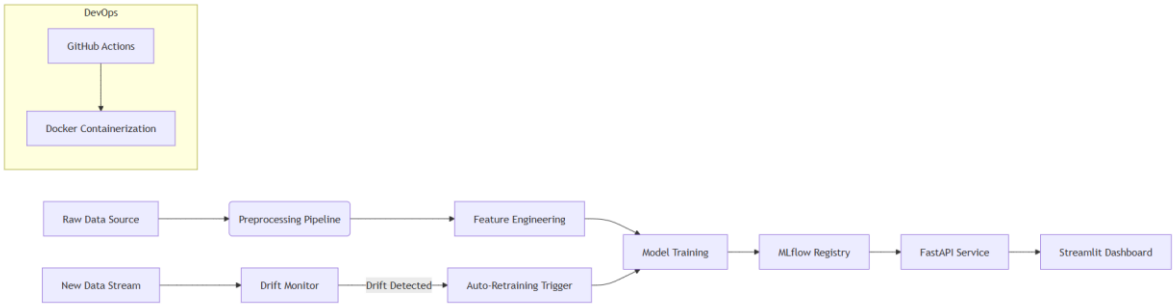

Figure 1: High-Level MLOps System Architecture

2.2 Architecture Description

- Ingestion Layer: Converts raw transactional data (CSV) into a structured format suitable for processing.
- Processing Layer: Modular Python scripts handle ETL (Extract, Transform, Load), feature extraction, and synthetic target generation.
- Training Layer: Utilizes Scikit-Learn for Random Forest regression, optimized via Grid Search.
- Serving Layer: FastAPI provides high-performance REST endpoints for real-time prediction.
- Observability Layer: Integrates MLflow for experiment tracking and custom logic for statistical data drift detection.

3. Data Methodology

3.1 Dataset Overview

The system utilizes the "Superstore Sales" dataset. As the original data lacked a direct profit target variable, a synthetic target (**Estimated_Profit**) was engineered to simulate regression analysis requirements.

Original Schema:

- Identifiers: **Row ID**, **Order ID**, **Customer ID**
- Temporal: **Order Date**, **Ship Date**
- Categorical: **Ship Mode**, **Segment**, **Region**, **Category**
- Numerical: **Sales**, **Postal Code**

3.2 Feature Engineering

To enhance predictive power, raw data was transformed into the following engineered features:

- Temporal Decomposition: Extraction of **Year**, **Month**, **Day**, and **WeekDay** from **Order Date** to capture seasonality.
- Business Logic: Calculation of **Discount** and **Profit_Margin**.
- Target Variable: **Estimated_Profit** generated based on sales heuristics.

3.3 Preprocessing Pipeline

The ETL process (**src/data/preprocess.py**) executes:

1. Data Cleaning: Handling missing values and correcting type inconsistencies.
2. Synthesis: Generating the target variable and derived features.
3. Storage: Persisting processed data to **data/processed/superstore_processed.csv**.

4. Machine Learning Lifecycle

4.1 Model Development

- Algorithm: Random Forest Regressor
    - *Rationale:* Chosen for its robustness against overfitting and ability to model non-linear relationships inherent in retail data.
- Input Features: **Sales**, **Discount**, **Year**, **Month**, **WeekDay**.
- Target: **Estimated_Profit**.

4.2 Hyperparameter Optimization

A Grid Search (**src/models/hyperparameter_tuning.py**) was conducted to maximize model generalization.

Optimal Configuration:

- **n_estimators**: 200
- **max_depth**: 5
- **min_samples_split**: 2

4.3 Model Evaluation

The model's performance was validated using a hold-out test set.

Metric	Value	Interpretation
R^2 Score	0.753	The model explains approximately 75.3% of the variance in profit.
MAE	15.62	On average, predictions deviate from actual profit by ~15.6 units.

4.4 Experiment Tracking

MLflow was integrated to ensure reproducibility and governance.

- Tracking URI: Local server simulating cloud registry.
- Artifacts: Model binaries (**model.pkl**), parameters, and performance metrics are versioned.

5. MLOps and Automation

5.1 Continuous Integration/Continuous Deployment (CI/CD)

Tool: GitHub Actions Workflow File: **.github/workflows/**

- Trigger: Push to **main** branch.
- Stages:
    1. Linting/Style Check: Ensures code quality.
    2. Dependency Installation: Sets up the Python environment.
    3. Automated Testing: Executes PyTest suite to validate API endpoints and model inference logic.

5.2 Automated Retraining & Drift Detection

To address "Concept Drift," the system implements an automated feedback loop.

Mechanism:

1. Drift Detection (**src/monitoring/drift_detection.py**):
    - Utilizes the Kolmogorov–Smirnov (K-S) statistical test to compare the distribution of incoming data streams against the reference training set.
    - Monitors critical features: **Sales**, **Discount**, and **Profit** proxies.
2. Retraining Trigger (**src/monitoring/auto_retrain.py**):
    - If the K-S statistic exceeds the threshold (p-value < 0.05), the system flags significant data drift.
    - Automatically initiates the training pipeline on the new data.
    - Registers the new model version in MLflow and updates the serving endpoint.

6. Application Deployment

6.1 Containerization (Docker)

The application is fully containerized to ensure consistency across development, staging, and production environments.

Components:

- **Dockerfile**: Defines the runtime environment (Python base, dependencies).
- **docker-compose.yml**: Orchestrates the multi-container setup (API and Dashboard).

6.2 Prediction Service (FastAPI)

Endpoint: **POST /predict** Description: Accepts JSON input for transaction features and returns real-time profit estimation.

Request Payload:

json

```
{
  "Sales": 500.0,
  "Discount": 0.2,
  "Year": 2026,
  "Month": 5,
  "WeekDay": 1
}
```

Response:

json

```
{
  "predicted_profit": 125.45
}
```

6.3 Business Intelligence Dashboard (Streamlit)

The dashboard has been successfully deployed to Streamlit Cloud for real-time access.

File: **src/dashboard/app.py** Provides a visual interface for stakeholders.

- KPI Cards: Real-time display of Total Sales, Average Sales, and Estimated Profit.
- Visualizations: Interactive plots for Category-wise sales, Monthly trends, and Correlation Heatmaps.
- What-If Analysis: UI forms allowing users to input hypothetical values to see predicted profit outcomes.

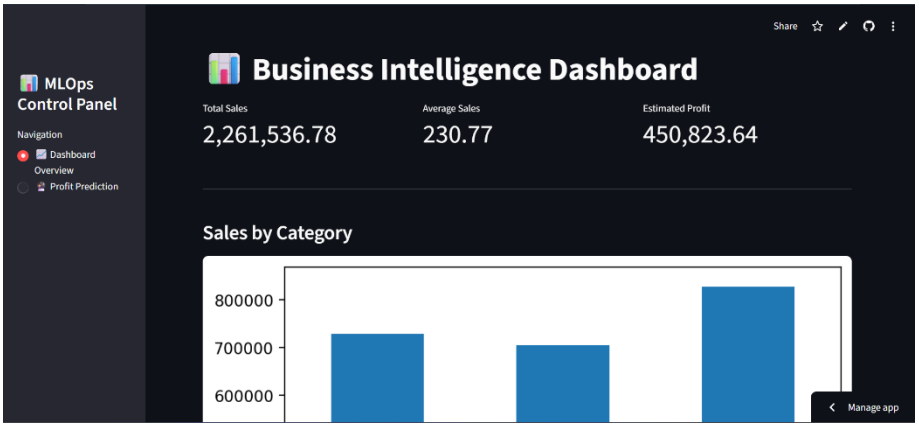

Figure 2: The deployed Streamlit Dashboard displaying KPIs and interactive sales visualizations.

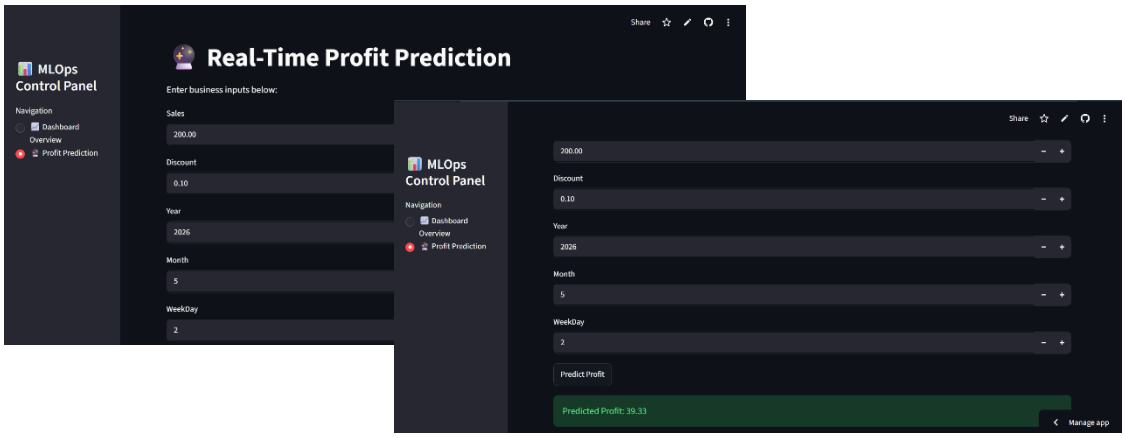

Figure 3: The "What-If" Analysis interface allowing real-time profit prediction inputs.

7. Testing Strategy

Framework: PyTest Directory: **tests/**

The testing suite ensures the reliability of core components:

- Unit Tests: Validation of data preprocessing functions and model loading mechanisms.
- Integration Tests: Verifying the **/predict** endpoint response status codes and output schemas.
- Result: 100% of tests passing in the CI/CD pipeline.

8. Operational Challenges & Mitigations

Challenge	Mitigation Strategy
Missing Target Variable	Implemented a synthetic feature generation module (**Estimated_Profit**) based on sales logic.
Date Formatting	Standardized datetime parsing during the preprocessing phase to handle multiple formats.
Localhost API Access	Configured Docker networking to allow the Streamlit container (Dashboard) to communicate with the FastAPI container (Service).
Dependency Conflicts	Utilized Virtual Environments and rigid **requirements.txt** pinning to resolve library version mismatches.

9. Future Scalability & Roadmap

While the current system is fully functional within a simulated environment, the following enhancements are proposed for enterprise-scale scaling:

1. Cloud Infrastructure: While the frontend dashboard is currently live on Streamlit Cloud, future work involves migrating the backend containers (FastAPI) from local Docker to managed services like AWS ECS or Google Kubernetes Engine (GKE) to ensure robust backend scalability and redundancy.
2. Feature Store: Implement a centralized Feature Store (e.g., Feast) to manage feature versions across training and serving.
3. Authentication: Add OAuth2/OpenID Connect to the FastAPI and Streamlit layers for role-based access control.
4. Real Profit Data: Replace the synthetic target with actual ERP data feeds for higher business value.
5. Advanced Monitoring: Integrate Prometheus and Grafana for deeper system-level metrics (latency, throughput, memory usage).

10. Conclusion

This project successfully delivered a comprehensive MLOps platform that bridges the gap between experimental data science and production engineering. By automating the retraining loop, containerizing the deployment, and implementing rigorous CI/CD practices, the system ensures high availability and adaptability. The architecture serves as a robust blueprint for enterprise-level AI implementation in the retail sector.